

Contents

SPECIFICATION — Specification Conformance Checker (<code>speccheck</code>; traceability graph, JUnit results, model-judged test strength; Python 3.12 + uv)	1
0. Intent and purpose	2
1. Actors and goals	3
2. Requirements (intent, high level)	4
3. Behavior and state model	6
3.1 Lifecycle	6
3.2 Executable flow	7
3.3 Durable artifacts	7
4. Interfaces / contracts	8
C-01 Spec ID grammar, declaration, citation, retirement	8
C-02 <code>SpecIndex</code>	9
C-03 <code>Citation</code> , <code>TestCase</code> , attribution	9
C-04 JUnit XML subset	10
C-05 Status algorithm and <code>IdStatus</code>	11
C-06 Judge interface	11
C-07 JSON report (<code>speccheck.json</code> , <code>schema_version</code> “1.0”)	12
C-08 Markdown report layout (<code>SPEC_CONFORMANCE_REPORT.md</code>)	14
C-09 LLM provider configuration	14
C-10 Judge instruction text (normative; F-004)	14
5. Interface specification	15
5.1 CLI (<code>speccheck</code>), the only surface	15
5.2 GUI	16
5.3 Diagnostics and verbosity contract	17
5.4 Exit codes	17
6. Invariants (must hold in every valid implementation)	17
7. Constraints (precise and measurable)	18
8. Edge cases and failure semantics	19
9. Acceptance criteria, tests, and evals	22
9.1 Extraction (C-01, C-02)	22
9.2 Citation and attribution (C-03)	23
9.3 Results mapping (C-04)	23
9.4 Status algorithm and metrics (C-05, C-07)	24
9.5 Judge contract (C-06)	24
9.6 Reports and determinism (C-07, C-08)	25
9.7 CLI, exit codes, diagnostics, boundary (§5)	26
9.8 Golden fixture (end to end)	27
9.9 Self-application (recorded, not gating)	28
9.10 Performance (recorded)	28
9.11 LLM judge evaluation (opt-in, <code>--judge llm</code> , not run in CI)	28
10. Dependencies and environment	29
11. Traceability matrix (id → where realized)	29
12. Open questions and decisions to confirm	32
Revision history	34

SPECIFICATION — Specification Conformance Checker (`speccheck`; traceability graph, JUnit results, model-judged test strength; Python 3.12 + uv)

- **Status:** v1.2 — v1.1 with D-07 partly resolved by the first T-49 runs: `max_tokens` raised from 400 to 4000 (C-06, T-33) so thinking models can finish the verdict JSON inside the

budget; the remaining §12 rows marked **confirm** are still decisions the build inherits from the author, not the requester

- **Language / stack:** Python 3.12 | standard library for the deterministic kernel (**re**, **ast**, **xml.etree**, **json**, **argparse**, **pathlib**) | CLI only; optional model-backed judge behind an `[llm]` extra
 - **Sources:** `one_sentence_prompt.md` (the brief); `../skills/spec-writing/SKILL.md` (the ID taxonomy and `SPEC.md` shape the checker consumes); `../skills/spec-build/SKILL.md` §3.17 (the manual conformance audit this tool automates); `../skills/spec-review/SKILL.md` §3.17 (the intent → requirement → contract → invariant → test → evidence chain); `../outline.md` Chapters 15–18 (where this system is the worked example); `SPEC_REVIEW_REPORT.md` (review of v0.1; F-001..F-017 below point at it); `FINAL_SPEC_REVIEW_REPORT.md` (review of v0.4; F-101..F-110 below point at it); `SPEC_v0.5_REVIEW_REPORT_by_QWEN.md` (independent review of v0.5 by a second model; its F-001..F-011 are cited below as Q-001..Q-011 to avoid collision)
 - **Scope of this document:** The deterministic conformance kernel (spec-ID extraction, citation graph, test-result mapping, status computation, reporting) and the contract around the optional model-backed *judge*. It does not specify the quality of the specification under check (`spec-review` owns that), does not specify how tests are run (results are consumed, not produced), and does not specify any semantic analysis of source code.
 - **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong recommendation; MAY = optional.
 - **Principle:** *The model may only ever make the news worse.* Every status is computed deterministically from evidence the operator can **grep**; the judge is permitted to downgrade a status with cited evidence, never to upgrade one, and every claim in the report points at a file and line.
-

0. Intent and purpose

`speccheck` answers one question about a project: **for every ID the specification declares, what evidence exists that the implementation realizes it?** It reads a `SPEC.md` written in the house ID taxonomy (R-nn requirements, C-nn contracts, I-nnn invariants, K-nn constraints, E-nn edge cases, T-nn tests), finds where each ID is cited in the source and test trees, joins those citations to the outcomes in a JUnit XML results file, and writes a conformance report in which every ID has exactly one status and every status has traceable evidence.

This is Phase 3 of `spec-build` (“re-read the spec and audit every artifact”) made mechanical. Humans doing that audit are slow and get tired; agents doing it are fast and confabulate. The design splits the work accordingly:

- The **deterministic kernel** does everything that can be done by pattern, graph, and arithmetic: which IDs exist, which are cited where, which tests ran and how they ended, what that makes each ID’s status, and what the coverage ratios are. Two runs on identical inputs produce byte-identical reports.
- The **judge** — a model-backed component, off by default — answers exactly one question per (test case, ID) edge: *does this test assert the observable behavior this ID describes, or does it merely execute code near it?* Its answer **MUST** cite lines inside the test; an answer without valid evidence is discarded as **UNKNOWN**. The judge can turn a **PASSING** into a **WEAKLY_PASSING**; it can never turn anything into **PASSING**.

Why this boundary (context the implementer should not re-derive): a checker that lets a model decide conformance has merely moved the vibe-coding problem one level up — the report becomes something to trust rather than something to verify. Keeping the model on the downgrade-only side of the line means a green report is as trustworthy as **grep** plus the test runner, and a yellow one carries a reason you can click on.

Non-goals (explicit, to constrain the solution space):

- No semantic analysis of source code — no type inference, control-flow, or “does this function implement R-07” reasoning, by the kernel *or* the judge. Source citations are literal token matches.
- No test execution. The checker consumes a results file; it never runs `pytest`, `go test`, or anything else.
- No spec-quality review (ambiguity, contradictions, missing sections). That is **spec-review**.
- No remediation. The checker reports; it never edits the spec, the code, or the tests.
- No multi-repository or multi-spec runs; one spec, one source tree set, one results file per invocation.
- No IDE integration, daemon mode, watch mode, or web UI.
- No language adapters beyond Python in v0.1 (see O-2); other languages get file-level attribution.
- The checker never reads its own outputs as inputs: the spec, the results file, and the two report files are excluded from every scan (C-03, F-002).

A known limitation, stated rather than hidden (F-013): citation is literal. A test that mentions [R-03] (#sec-requirements) as *data* — asserting on a report that contains it, or on an error message — cites R-03 exactly as a test that proves it does. The `speccheck:ignore` markers in C-01 are the opt-out; they are the author’s responsibility, and `speccheck` never infers intent from context.

Relationship to the skills: `spec-writing` produces the input; `spec-review` grades it; `spec-build` builds from it and, in Phase 3, does by hand what this tool does by machine. The report this tool writes is intended to be pasted into `SPEC_BUILD_REPORT.md` as the per-ID evidence table.

1. Actors and goals

Actor	Goals
Operator (human at a terminal, or a CI job)	Run one command against a project and get a report plus an exit code that can gate a merge. Never has to trust a number without a path to its evidence.
Extractor (<code>speccheck/extract.py</code>)	Turn <code>SPEC.md</code> into the set of declared IDs (with statement text, family, retired flag) and turn source/test files into citations, deterministically.
Attributor (<code>speccheck/attribute.py</code>)	Map each citation in a test file to the test case that contains it (Python adapter), falling back to file-level attribution for anything it cannot parse.
Results Mapper (<code>speccheck/results.py</code>)	Read a JUnit XML file and join each <code><testcase></code> to an attributed test case, yielding outcomes per test case.
Grapher (<code>speccheck/graph.py</code>)	Build the <code>ID → source-file / ID → test-case</code> edge sets and compute each ID’s deterministic status and the coverage metrics.
Judge (<code>speccheck/judge.py</code> ; providers <code>judge_mock.py</code> , <code>judge_llm.py</code>)	For each (test case, ID) edge of a PASSING ID, return a verdict with evidence lines; the mock provider is deterministic, the LLM provider is opt-in. Single-principal: the judge sees one edge at a time and holds no state across calls.
Reporter (<code>speccheck/report.py</code>)	Render the Markdown and JSON reports from the graph and judge results, in a fixed order, with no content that varies between identical runs.
CLI (<code>speccheck/cli.py</code> , entry point <code>speccheck</code>)	The only surface. Parse arguments, wire the pipeline, apply the exit-code and verbosity contracts.

2. Requirements (intent, high level)

ID	Statement	Source
R-01	The checker MUST read a Markdown specification and extract every <i>declared</i> spec ID together with its family and statement text, per the grammar in C-01 .	prompt; spec-writing §ID taxonomy
R-02	The checker MUST recognize a declared ID as <i>retired</i> when its declaration is struck through (<code>~~R-07~~</code>), and MUST exclude retired IDs from coverage denominators.	spec-writing §ID taxonomy (“retire old ones with a strike-through”)
R-03	The checker MUST scan every text file under each <code>--src</code> root, except the files C-03 excludes, and record a <i>source citation</i> for each (file, ID) pair where the ID token occurs, per C-03 .	prompt (“analyzes an implementation”)
R-04	The checker MUST scan every text file under each <code>--tests</code> root, except the files C-03 excludes, and record a <i>test citation</i> for each (test case, ID) pair, attributing the citation to the enclosing test case when the file’s language adapter can delimit test cases, and to the file otherwise.	prompt (“and its tests”)
R-05	When <code>--results</code> is given, the checker MUST read a JUnit XML file and join each <code><testcase></code> to an attributed test case, recording the outcome passed , failed , error , or skipped per C-04 .	spec-build Phase 1 exit gate
R-06	The checker MUST assign every declared, non-retired ID exactly one deterministic status from the set in C-05 , computed by the algorithm in C-05 and by nothing else.	spec-build Phase 3.1
R-07	The checker MUST list every <i>dangling citation</i> : an ID token cited in a source or test file that is not declared in the specification.	spec-build Phase 3.1 (“no ID is dangling”)
R-08	The checker MUST list every <i>stale citation</i> : a citation of a retired ID.	R-02
R-09	The checker MUST compute the coverage metrics in C-07 §metrics, applying the zero-denominator rules stated there; it MUST NOT raise or emit 0.0 for an undefined ratio.	spec-writing §precision rules
R-10	When <code>--judge</code> is <code>mock</code> or <code>llm</code> , the checker MUST obtain, for each (test case, ID) edge whose ID is PASSING and whose test outcome is passed (F-104), one judge verdict from the set ASSERTS , EXECUTES_ONLY , UNRELATED , UNKNOWN , with evidence per C-06 .	prompt (“AI-assisted”)
R-11	A judge verdict MUST only ever lower an ID’s status (PASSING → WEAKLY_PASSING); the judge MUST NOT change any status other than PASSING and MUST NOT create, remove, or re-attribute citations.	§0 principle
R-12	The checker MUST write a Markdown report (<code>SPEC_CONFORMANCE_REPORT.md</code>) laid out per C-08 in which every declared ID appears exactly once.	prompt (“traceable conformance report”)
R-13	The checker MUST write a JSON report (<code>speccheck.json</code>) conforming to C-07 that contains everything the Markdown report shows.	CI consumption

ID	Statement	Source
R-14	The checker MUST exit with the codes in §5.4, and the exit code MUST be a pure function of the JSON report content plus the <code>--strict</code> flag.	CI gating
R-15	With <code>--strict</code> , any in-scope ID whose status is not <code>PASSING</code> , any dangling citation, and any stale citation MUST cause exit 1. Status (C-05) is the only input to this rule; individual judge verdicts are not consulted again (F-003).	spec-build “done when”
R-16	With <code>--judge none</code> or <code>--judge mock</code> , two runs over byte-identical inputs MUST produce byte-identical <code>SPEC_CONFORMANCE_REPORT.md</code> and <code>speccheck.json</code> .	spec-writing §precision rules (determinism)
R-17	The checker MUST implement the diagnostics contract in §5.3: silent by default; <code>--verbose/--verbose INFO</code> emit metadata only; <code>--verbose DEBUG</code> additionally emits judge prompts and responses; all diagnostics go to stderr and never alter stdout or the report files.	spec-writing §5 cross-cutting contracts
R-18	With <code>--judge none</code> or <code>--judge mock</code> , the process MUST perform no network I/O, and <code>speccheck --self-check</code> MUST verify this boundary.	§0 boundary
R-19	The checker MUST treat all inputs as read-only; it MUST NOT create, modify, or delete any file outside <code>--out</code> , except as I-001 permits for <code>--self-check</code> (F-104).	safety
R-20	All paths in reports MUST be relative to <code>--root</code> (default: current directory) and use <code>/</code> as separator regardless of host OS.	R-16 (portability of golden reports)
R-21	On completion (exit 0 or 1) the checker MUST print exactly one summary line to stdout in the format of §5.1 and nothing else.	CI log hygiene
R-22	The mock judge MUST be deterministic and MUST NOT require configuration: it returns <code>ASSERTS</code> when the test case’s span contains an assertion token per C-06 , else <code>EXECUTES_ONLY</code> .	R-16 ; testability of the judge path
R-23	The LLM judge MUST read its endpoint, model name, and API key from environment variables per C-09 ; a missing variable MUST be a usage error (exit 2), and the key MUST never appear in any output, log, or report at any verbosity.	secret handling
R-24	Every deterministic status (C-05 steps 1–4), every count, and every metric in the report MUST be reproducible by an operator using only the report’s own evidence table, <code>grep</code> , and the results file; <code>WEAKLY_PASSING</code> and the judge-derived metrics MUST be reproducible from the same inputs plus the verdicts <i>as recorded</i> in the report (F-017).	prompt (“traceable”)
R-25	The checker MUST treat family T per C-05 : a T id’s status is determined by test citations and their outcomes only; source citations of a T id are recorded as evidence but never change its status, and a T id with no test citation is <code>UNCITED</code> , never <code>UNTESTED</code> (F-001).	spec-build (“every T-nn cites its spec ID”)

ID	Statement	Source
R-26	The LLM judge MUST use exactly the request body, response path, and instruction text pinned in C-06 and C-10 , and the report MUST record the SHA-256 of the instruction text used (F-004).	review F-004
R-27	The checker MUST honor the ignore markers in C-01 : a line containing <code>speccheck:ignore</code> yields no citations, and a file whose first three lines contain <code>speccheck:ignore-file</code> is not scanned; ignored files are counted in a Note (F-013).	review F-013
R-28	With <code>--strict</code> and <code>--judge llm</code> , the checker MUST exit 1 when the judge was unavailable (<code>judge_available == false</code>) or when <code>unknown_rate</code> exceeds <code>--max-unknown</code> (K-11), and the summary line MUST name the reason (F-012).	review F-012
R-29	The summary line MUST be a single line of ASCII text written to stdout as UTF-8 regardless of locale, in the exact format of §5.1 (F-010).	review F-010

3. Behavior and state model

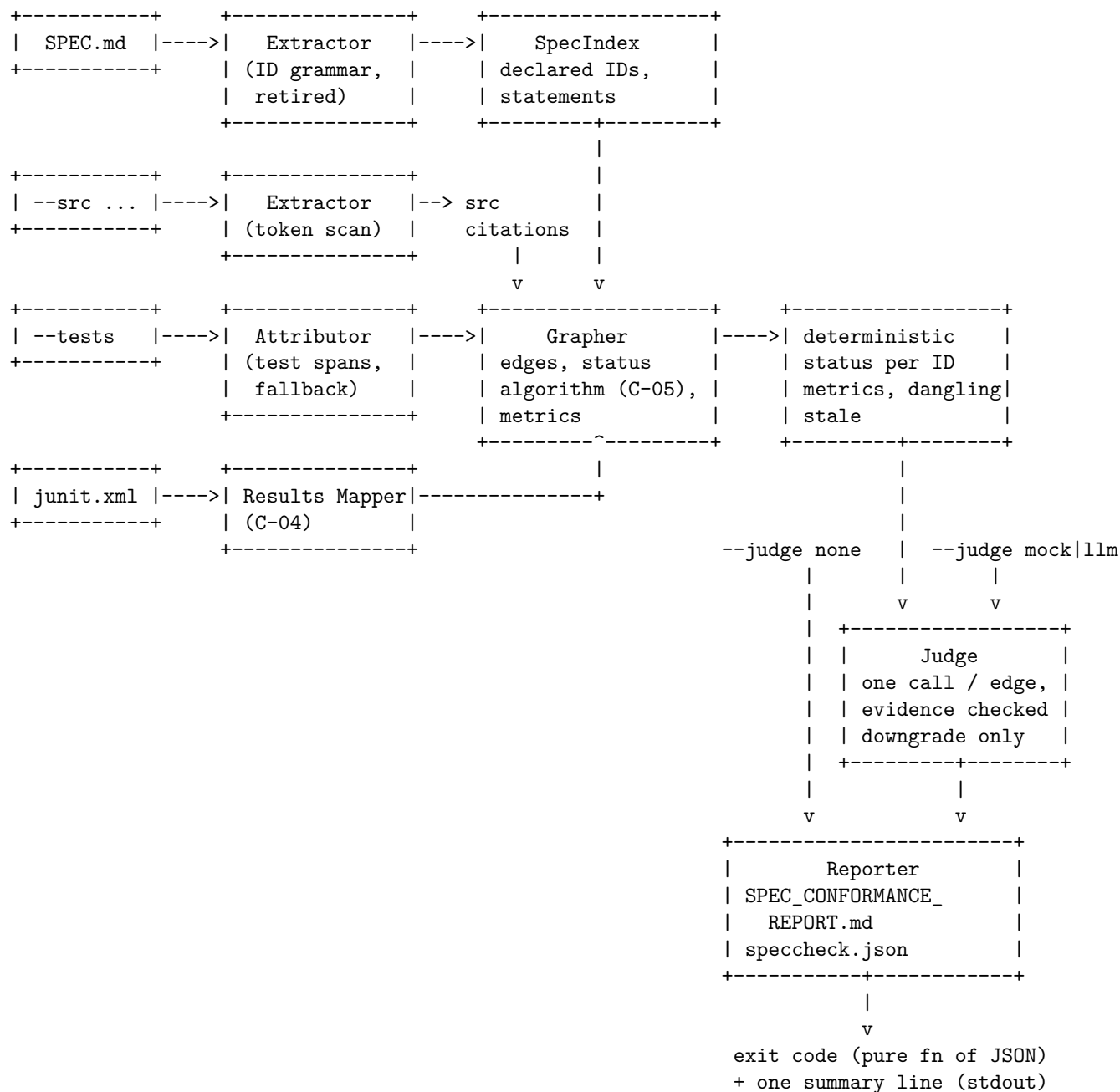
3.1 Lifecycle

A run is a single, stateless pipeline. There is no persistent state between runs, no cache, and no partial-output mode: either both report files are written or neither is. The Reporter guarantees this by rendering both documents in memory, then, in this order (F-106): (1) delete any leftover `<out>/speccheck.json.*.tmp` and `<out>/SPEC_CONFORMANCE_REPORT.md.*.tmp` from earlier runs; (2) write `speccheck.json.<nonce>.tmp` and `SPEC_CONFORMANCE_REPORT.md.<nonce>.tmp`, where `<nonce>` is 8 random hex digits chosen per run (F-103); (3) rename the JSON temporary to `speccheck.json`; (4) rename the Markdown temporary to `SPEC_CONFORMANCE_REPORT.md`. If any step fails, every temporary and any file this run already renamed is removed before exit 3 (F-015). The nonce never appears in either report, so [I-002](#) is unaffected.

Stage	Owner	Input	Output	Fails with
parse-args	CLI	argv, env	Config	exit 2
extract-spec	Extractor	SPEC.md	SpecIndex (C-02)	exit 3 (E-01 , E-02 , E-03)
scan-src	Extractor	<code>--src</code> roots minus C-03 exclusions	Citation[] (C-03)	never (E-10 , E-11 are notes)
scan-tests + attribute	Extractor, Attributor	<code>--tests</code> roots	TestCase[] , Citation[]	never (E-12 is a note)
map-results	Results Mapper	<code>--results</code>	Outcome[] (C-04)	exit 3 (E-05)
graph + status	Grapher	all of the above	IdRecord[] with deterministic status; dangling/stale lists; metrics	never
judge (optional)	Judge	PASSING edges	Verdict[] (C-06)	never (E-14 .. E-17 yield UNKNOWN)
report	Reporter	everything	SPEC_CONFORMANCE_REPORT.md and speccheck.json	exit 3 if out unwritable

Stage	Owner	Input	Output	Fails with
exit	CLI	JSON report, <code>--strict</code>	exit code (§5.4), summary line	—

3.2 Executable flow



3.3 Durable artifacts

Artifact	Written to	Version	Shape
SPEC_CONFORMANCE_REPORT.md	md-out (default .)	mirrors JSON	C-08
speccheck.json	--out	schema_version schema_version: "1.0"	C-07

Both files are overwritten on every successful run, via the temp-and-rename sequence in §3.1. No other file is ever written, and no temporary survives a run (I-001).

4. Interfaces / contracts

C-01 Spec ID grammar, declaration, citation, retirement

```
ID      := FAMILY "-" DIGITS
FAMILY  := "R" | "C" | "I" | "K" | "E" | "T"          # 0-n, D-nn and F-nnn are NOT conformance IDs
DIGITS  := [0-9]{1,3}
TOKEN   := ID bounded by non-alphanumerics on both sides # regex: (?![A-Za-z0-9])([RCIKET])-([0-9])
```

Declaration (in SPEC.md only) - an ID is DECLARED when, outside fenced code blocks, either form occurs (F-009). A fenced code block (F-108) opens at a line whose first non-space characters are three backticks (U+0060 x3) or three tildes (~~~), optionally followed by an info string, and closes at the next line whose first non-space characters are the same three-character marker - the same character as the opener - followed by nothing but whitespace; a closing line's info string is not permitted, a backtick fence is never closed by tildes nor a tilde fence by backticks, and an unclosed fence runs to end of file (Q-008).

```
(a) table row:    | **R-07** | <statement> | ...
row              := a line whose first non-space character is "|"
cells            := the row split on every "|" that is not preceded by "\" and not inside a
                  backtick span (`...`); the leading and trailing empty cells are dropped
the ID MUST be the ENTIRE trimmed content of the FIRST cell, in one of the forms
  **ID**   ~~~~ID~~~   ~~~~ID~~~
statement := the trimmed text of the SECOND cell ("" if there is none)
A bold ID in any other cell is not a declaration (E-31). A separator row (cells made of
  "-" and ":" only) is never a declaration.
```

```
(b) heading:      ### C-03 <statement>
the ID token (optionally wrapped ~ ~ ~) MUST be the first token after the "#" run;
statement := the rest of the heading text, trimmed. "### 9.1 Extraction (C-01, C-02)"
declares nothing.
```

A SECOND declaration of the same ID is E-02 (exit 3); the message names the ID and both lines. Nothing "wins": the run produces no report (Q-006).

Retirement - a declaration whose ID token is wrapped in ~ ~ ~ :

```
| ~~~~R-07~~~ | ... | or | ~~~~R-07~~~ | ... or ### ~C-03~ ...
```

A retired ID is DECLARED and RETIRED. Retirement is not un-done by a later plain declaration (E-03).

Citation (in --src / --tests files) - every TOKEN occurrence in a text file is a citation, regardless of comments, strings, or code. Fenced code blocks are NOT excluded in source files (only in SPEC.md). ID numbers are compared numerically: R-7 and R-07 and R-007 are the same ID.

Ignore markers (F-013), matched literally and case-sensitively:

```
speccheck:ignore      anywhere on a line -> that line yields no citations (the line still
                                counts toward a test case's span)
```

speccheck:ignore-file anywhere in the first 3 lines of a file -> the file is not scanned at all (no citations, no test cases); one Note per run: "ignored N file(s) by speccheck:ignore-file"

The markers have no effect inside SPEC.md.

Normalized form used everywhere in output: FAMILY "-" zero-padded to 2 digits for R/C/K/E/T, 3 digits for I (R-07, I-005). An ID declared as R-7 is reported as R-07.

Family T (F-001): a T id names an acceptance test. It is IN SCOPE and is expected to be cited by the test case that realizes it (in that case's docstring or a comment). Source citations of a T id are recorded but do not affect its status (C-05 step 2b). All six families count toward `in\_scope` and `conformance`.

C-02 SpecIndex

```
@dataclass(frozen=True)
```

```
class SpecId:
```

```
    family: str          # one of "R","C","I","K","E","T"
```

```
    number: int          # 0..999
```

```
    text: str            # statement (table cell or heading remainder), whitespace-collapsed, may be ""
```

```
    line: int            # 1-based line of the declaration in SPEC.md
```

```
    retired: bool
```

```
@dataclass(frozen=True)
```

```
class SpecIndex:
```

```
    path: str            # relative, POSIX
```

```
    ids: tuple[SpecId, ...] # sorted by (family order R,C,I,K,E,T ; number)
```

```
    # invariant: no two SpecId share (family, number)
```

C-03 Citation, TestCase, attribution

```
@dataclass(frozen=True)
```

```
class TestCase:
```

```
    file: str            # relative, POSIX
```

```
    name: str            # function name, e.g. "test_zero_rate"; "" for file-level
```

```
    classname: str       # dotted module path used to match JUnit classname, e.g. "tests.test_core"
```

```
    start: int           # 1-based first line of the span; 1 for file-level
```

```
    end: int             # inclusive last line; last line of file for file-level
```

```
@dataclass(frozen=True)
```

```
class Citation:
```

```
    id: str              # normalized ID
```

```
    file: str
```

```
    line: int
```

```
    kind: str            # "src" | "test"
```

```
    testcase: TestCase | None # for kind=="test": the enclosing case, or the file-level case
```

Python adapter (files ending .py under a --tests root):

```
* parse with `ast`; a test case is (F-007):
```

- every top-level FunctionDef or AsyncFunctionDef whose name starts with "test_";
- every FunctionDef/AsyncFunctionDef whose name starts with "test" (underscore NOT required: "testAddition" and "test_addition" both qualify; F-102) that is a direct member of a top-level ClassDef which is RECOGNIZED: its name starts with "Test", OR any of its bases is a Name/Attribute whose final identifier ends with "TestCase";

```
span = node.lineno .. node.end_lineno (decorators included, per `ast`).
```

- * a "test*" function that is a member of an UNRECOGNIZED top-level class, or of a nested class, is not a test case; its lines belong to the file-level case, and the file gets one Note: "undelimited tests in <path>: <Class.name>, ..." (E-28).
- * classname = the file path relative to --root with "/" -> "." and ".py" dropped ("tests/test_core.py" -> "tests.test_core"); methods append "." + ClassName.
- * a citation on a line inside a span is attributed to that case; any other citation in the file is attributed to the file-level case (name "").
- * a .py file that fails to parse -> whole file is file-level (E-12; a note, not an error).

Fallback adapter (any other text file): one file-level case per file; its classname is derived exactly as for Python (path relative to --root, "/" -> ".", final extension dropped; F-108).

Text vs binary (F-008): a file is BINARY if any of its first 8192 bytes is 0x00. Binary files are skipped silently (no Note, no citations, no test case). Everything else is text and is decoded as UTF-8 with errors="replace" (E-11). Symbolic links - to files or to directories - ENCOUNTERED DURING DESCENT are never followed (K-03); one Note per run if any were skipped (E-30). A path given on the command line (--spec, --src, --tests, --results, --out, --root) is resolved once, checked against --root (E-09), and then used in its resolved form: a symlinked --src root is descended, and it is not counted as a skipped symlink (Q-007).

Excluded from every scan (F-002), whether or not they lie under a --src/--tests root, and whether or not they exist yet: the file named by --spec, the file named by --results, <out>/SPEC_CONFORMANCE_REPORT.md, <out>/speccheck.json, and every file directly under <out> whose name starts with ".speccheck.json." or ".SPEC_CONFORMANCE_REPORT.md." and ends with ".tmp" (the §3.1 temporaries, including leftovers from a killed run; F-103). Exclusion is by resolved path (symlinks resolved), is silent (no Note), and applies before K-02/K-03 filtering.

C-04 JUnit XML subset

Root: <testsuites> or <testsuite>. Every <testcase> element anywhere below the root is an outcome.

classname := @classname (may be ""); an empty classname ends every dotted name on a component boundary, so it matches every TestCase with the same join_name, and the longest-suffix rule below then applies with suffix length 0 - F-105)

name := @name (required; missing -> E-05)

outcome := "failed" if a <failure> child exists
 | "error" if an <error> child exists
 | "skipped" if a <skipped> child exists
 | "passed" otherwise

Join to a TestCase (F-006): a result joins the TestCase whose `join\_name` equals the result's join_name AND whose dotted `classname` ends with the result's dotted classname on a component boundary ("tests.test_core" matches "tests.test_core" and "test_core"; it does not match "unit_test_core"). Among candidates, the one whose classname has the LONGEST common dotted suffix with the result wins; if two or more candidates tie, the result is UNATTRIBUTED and one Note names the result and every candidate (E-27). `join\_name` is defined as:

```
join_name := if name ends with "]" and contains "[": name[:name.index("[")]
            else:
                name
                i.e. everything from the FIRST "[" to the end is removed, whatever it contains
                (F-005, F-101): "test_x[3-True]" -> "test_x"; "test_x[list[int]]" -> "test_x";
                "test_y[a][b]" -> "test_y".
```

param := the removed text without its outer brackets ("3-True", "list[int]", "a)[b]", or null.

Every <testcase> that joins the same TestCase (parametrized variants, or duplicates) is kept in that case's `results` list; the case's single `outcome` = worst of them, worst order error > failed > skipped > passed (E-06, E-24).

A <testcase> that joins no TestCase is an UNATTRIBUTED result (E-07): listed, never counted.

Malformed XML, or a root that is neither element: E-05 -> exit 3.

C-05 Status algorithm and IdStatus

IdStatus (deterministic): RETIRED | UNCITED | UNTESTED | UNVERIFIED | FAILING | SKIPPED | PASSING
IdStatus (judge-only): WEAKLY_PASSING

For each declared ID x:

1. if x.retired -> RETIRED (excluded from denominators)
2. T := { test cases with >=1 test citation of x } # file-level cases included
S := { source files with >=1 src citation of x }
 - 2a. if x.family != "T":
if T == {} and S == {} -> UNCITED
if T == {} -> UNTESTED
 - 2b. if x.family == "T": # F-001: S is evidence only
if T == {} -> UNCITED
3. if no --results given, or no case in T has an outcome -> UNVERIFIED
4. O := outcomes of cases in T that have one
if any o in O is failed or error -> FAILING
elif every o in O is skipped -> SKIPPED
else -> PASSING
5. (judge enabled only) if status == PASSING:
V := verdicts for edges (t, x), t in T with outcome passed
if V is non-empty and no v in V is ASSERTS and some v in V is EXECUTES_ONLY or UNRELATED -> WEAKLY_PASSING
else -> PASSING (UNKNOWN never changes status)

Cases in T without an outcome are reported as "unrun" for x and ignored in step 4.

The judge is never consulted for edges whose ID is not PASSING after step 4 (I-010).

Step 5 is the ONLY place verdicts influence anything: an ID with verdicts {ASSERTS, EXECUTES_ONLY} is PASSING, and PASSING is what --strict tests (R-15, §5.4; F-003). There is no per-edge rule anywhere else in this document.

C-06 Judge interface

```
@dataclass(frozen=True)
class JudgeRequest:
    id: str # "R-07"
    statement: str # SpecId.text
    testcase: TestCase
    source: str # lines testcase.start..testcase.end, each prefixed "<lineno>\t" (F-109)

@dataclass(frozen=True)
class Evidence:
    file: str
    line: int # MUST satisfy testcase.start <= line <= testcase.end

@dataclass(frozen=True)
class Verdict:
    verdict: str # "ASSERTS" | "EXECUTES_ONLY" | "UNRELATED" | "UNKNOWN"
    evidence: tuple[Evidence, ...]
    rationale: str # <= 280 characters, single line

class Judge(Protocol):
    def judge(self, req: JudgeRequest) -> Verdict: ...
```

Validation applied by the kernel to EVERY provider's raw answer (judge.py, not the provider):

```

* verdict not in the four-value set -> UNKNOWN
* verdict == ASSERTS and evidence is empty -> UNKNOWN
* any evidence.file != testcase.file, or line outside span -> UNKNOWN
* rationale longer than 280 chars -> truncated to 277 + "..."
* provider raised, timed out (K-05), or returned non-JSON -> UNKNOWN, rationale = "judge: <class
An UNKNOWN produced by validation is recorded with `coerced: true` in the JSON report.

```

LLM provider wire format (judge_llm.py; F-004) - an OpenAI-compatible chat-completions endpoint:

```

Request: POST SPECHECK_JUDGE_URL
Headers: Content-Type: application/json ; Authorization: Bearer <SPECHECK_JUDGE_API_KEY>
Body (JSON, exactly these keys):
{ "model": <SPECHECK_JUDGE_MODEL>,
  "temperature": 0,
  "max_tokens": 4000,
  "messages": [
    { "role": "system", "content": <the C-10 instruction text, verbatim> },
    { "role": "user", "content": <JSON object {id, statement, file, start, end, source}
                                source = the span with every line prefixed by its
                                1-based line number and one TAB (F-109)> } ] }

```

Response: HTTP 200 with a JSON body; the model's text is read from choices[0].message.content .
Any other status, a non-JSON body, or a missing path -> E-14 / E-15.
The text is stripped of surrounding whitespace and of ONE enclosing ``` or ```json fence
if present, then MUST parse as a single JSON object {verdict, evidence:[{file,line}],
rationale}; anything else is non-JSON (-> UNKNOWN, E-15).

Exactly one HTTP request per edge; no retries (K-06). The SHA-256 of the C-10 text as sent is
recorded in the report as `judge\_prompt\_sha256` (C-07).

Mock provider (judge_mock.py):

```

verdict = ASSERTS if the span contains an assertion token, else EXECUTES_ONLY
assertion token := a line matching `^s*assert\b` or containing .assert or pytest.raises(
evidence = every such line (file, line); rationale = "mock: assertion token on N line(s)" / "mock: no

```

C-07 JSON report (speccheck.json, schema_version "1.0")

```

{
  "schema_version": "1.0",
  "spec": "SPEC.md",
  "judge": "none | mock | llm",
  "judge_available": null,
  "judge_prompt_sha256": "<64 hex chars; present only when judge == llm>",
  "strict": false,
  "max_unknown": 0.2,
  "strict_judge_failure": null,
  "ids": [
    {
      "id": "R-07",
      "family": "R",
      "statement": "...",
      "line": 88,
      "status": "PASSING",
      "src": [ {"file": "src/pkg/core.py", "lines": [12, 40]} ],
      "tests": [
        {
          "file": "tests/test_core.py", "name": "test_zero_rate", "classname": "tests.test_core",

```

```

    "lines": [17], "outcome": "passed",
    "results": [ {"name": "test_zero_rate[0]", "param": "0", "outcome": "passed"},
                  {"name": "test_zero_rate[1]", "param": "1", "outcome": "passed"} ],
    "verdict": {"verdict": "ASSERTS", "evidence": [{"file": "tests/test_core.py", "line": 21}],
                "rationale": "...", "coerced": false} // null when the case was not judged (Q-001)
  },
  ],
  "unrun": [ {"file": "...", "name": "..."} ]
}
],
"dangling": [ {"id": "R-99", "file": "src/pkg/x.py", "line": 3} ],
"stale":     [ {"id": "R-02", "file": "tests/test_old.py", "line": 9} ],
"unattributed_results": [ {"classname": "tests.test_gone", "name": "test_x", "outcome": "passed"} ],
"notes": [ "skipped 1 file over 2 MiB: data/big.json", "parse fallback: tests/test_bad.py" ],
"metrics": {
  "declared": 61, "retired": 2, "in_scope": 59,
  "by_status": {"PASSING": 50, "WEAKLY_PASSING": 3, "FAILING": 1, "SKIPPED": 0, "UNVERIFIED": 2, "UNKNOWN": 1},
  "conformance_ratio": "50/59", "conformance": 0.8475,
  "by_family": { "R": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "C": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "I": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "K": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "E": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "T": {"in_scope": 24, "passing": 22, "ratio": 0.9167} },
  "judge_strength_ratio": "50/53", "judge_strength": 0.9434,
  "unknown_rate": 0.0
},
"exit_code": 1
}

```

Rules: keys emitted in exactly this order (`judge\_prompt\_sha256` omitted entirely unless judge == llm; `max\_unknown` always present, echoing K-11; `strict\_judge\_failure` is null, "unavailable", or "unknown_rate" and is non-null only when R-28 forced exit 1 - when BOTH reasons hold, "unavailable" is recorded, since a fully unavailable judge makes the unknown rate vacuous; Q-004); `ids` sorted by (family order R,C,I,K,E,T ; number); `results` sorted by name; `src`/`tests`/`dangling`/`stale`/`unattributed\_results` sorted by (file, line) then name; `notes` is a list of strings sorted ascending by Unicode code point (each Note's text is fixed by the E-case that produces it, so this order is total and reproducible; Q-002); `lines` ascending, unique. No timestamps, hostnames, absolute paths, durations, or version strings other than schema_version (R-16).

Verdict field (Q-001): every `tests[]` entry has the key `verdict`. It is a verdict object when the case was judged (judge enabled, ID `PASSING` after C-05 step 4, outcome `passed`) and `null` otherwise - including every entry under `--judge none`. The key is never omitted.

Names (Q-011): `tests[].name` is the `TestCase.name` - `` for a file-level case. `(file)` is a Markdown-only rendering (C-08) and never appears in the JSON. R-13's "contains everything" concerns data; a rendering label is not data.

Numbers (Q-009): every ratio is computed as $\text{Decimal}(\text{numerator}) / \text{Decimal}(\text{denominator})$ under a 28-digit context and quantized to 4 places with ROUND_HALF_EVEN; it is emitted as a JSON number with exactly four decimal places (0.9000, not 0.9), so the JSON text is byte-determined without reference to binary floating point. K-11 compares these quantized values. `` in the summary line is the quantized `conformance` times 100, quantized to 1 place with ROUND_HALF_EVEN.

Metrics:

```

conformance      = |PASSING| / in_scope                                (WEAKLY_PASSING is NOT passing)
by_family[f].ratio = |PASSING in f| / |in_scope in f|
judge_strength    = |PASSING| / (|PASSING| + |WEAKLY_PASSING|)          (judge enabled only; else absent)
unknown_rate      = |edges with verdict UNKNOWN| / |judged edges|      (judge enabled only; else absent)

```

Zero denominators: ratio = null (JSON) / "n/a" (Markdown); the "a/b" string is still emitted ("0/0"). in_scope == 0 is E-01 (exit 3), so `conformance` itself is always defined on exit 0/1.

Shape rules (F-016): `by\_family` ALWAYS has exactly the six keys R, C, I, K, E, T in that order,

each with in_scope/passing/ratio, even when in_scope is 0. `by\_status` ALWAYS has exactly the seven in-scope statuses in the order shown, each a count (0 allowed); RETIRED is never a key. `judge\_available` is null when judge == none; with mock or llm it is true when at least one judge call succeeded OR no edge was eligible (vacuously available), and false only when every call failed (E-14). The C-07 example's `// comments` are explanatory and never emitted. Family T counts toward in_scope and conformance like every other family (F-001).

C-08 Markdown report layout (SPEC_CONFORMANCE_REPORT.md)

Specification Conformance Report

****Spec:**** `SPEC.md` · ****Judge:**** none|mock|llm (available|unavailable) · ****Strict:**** on|off

1. Verdict

<one line: the §5.1 summary line, verbatim>

2. Metrics

Metric Value	# conformance a/b (0.xxxx); by-status counts; per-family table;
	# judge_strength and unknown_rate rows only when judge enabled

3. Per-ID evidence

ID Status Statement Source citations Test citations (outcome · verdict)

one row per declared ID in C-07 order; RETIRED rows included, with the ID cell (only) struck through:
citations rendered as `file:line`; verdict rendered as ASSERTS / EXECUTES_ONLY / UNRELATED / UNKNOWN(
or as an em dash (-) when the JSON `verdict` is null (Q-001); a file-level case is rendered with the
label `(file)` in place of its empty name (Q-011)

## 4. Dangling citations	# table or "None."
## 5. Stale citations	# table or "None."
## 6. Unattributed results	# table or "None."
## 7. Unrun test citations	# table (ID, file, name) or "None."
## 8. Judge details	# present only when judge enabled: one row per judged edge (verdict non-n
## 9. Notes	# bullet list or "None."

C-09 LLM provider configuration

SPECHECK_JUDGE_URL required with --judge llm e.g. https://host/v1/chat/completions
SPECHECK_JUDGE_MODEL required with --judge llm model identifier passed through verbatim
SPECHECK_JUDGE_API_KEY required with --judge llm sent as "Authorization: Bearer <key>"
SPECHECK_JUDGE_TIMEOUT optional, seconds, default 30, integer 1..300 (else exit 2)
The URL and model MAY appear at INFO; the key MUST NOT appear anywhere, including DEBUG output
and error messages (redact to "***").

C-10 Judge instruction text (normative; F-004)

The file speccheck/judge_prompt.md is shipped as package data and its content is exactly the text below (trailing newline, \n line endings). It is sent verbatim as the system message (C-06). Changing it is a spec change: bump this document's version and the file together.

You are a test-strength judge for a specification conformance checker.

You will receive one JSON object with these fields:

id	- a specification ID, e.g. "R-07"
statement	- the normative text of that ID
file	- the path of one test file

start - the first line number of one test case in that file
 end - the last line number of that test case (inclusive)
 source - the text of lines start..end, one line per source line, each line prefixed by its
 absolute line number and a tab (e.g. "17<TAB>assert x == 2"); cite those numbers

Answer exactly one question: does this test case ASSERT the observable behavior described by the statement, or does it merely execute code near it?

Reply with one JSON object and nothing else - no prose, no markdown fence:

```
{"verdict": <VERDICT>, "evidence": [{"file": <file>, "line": <n>}, ...], "rationale": <text>}
```

VERDICT is exactly one of:

"ASSERTS" - the test contains at least one assertion (assert statement, assertion method, expected-exception context, or equivalent) whose expected value or condition corresponds to what the statement requires. You MUST list every such assertion line in evidence.
 "EXECUTES_ONLY" - the test runs code related to the statement but no assertion checks the behavior the statement requires (assertions absent, trivial, or unrelated).
 "UNRELATED" - the test does not exercise the behavior the statement describes at all.
 "UNKNOWN" - you cannot decide from the source given. Prefer UNKNOWN over guessing.

Rules:

- Every evidence line number MUST be between start and end inclusive, in the given file.
- Cite only lines that exist in source. Do not invent lines.
- rationale is one sentence, at most 280 characters.
- Judge only the given test case. Do not assume what other tests do.
- A test that mentions the ID in a comment or string is not evidence of asserting it.

5. Interface specification

5.1 CLI (speccheck), the only surface

```
speccheck check --spec SPEC.md [--src DIR]... [--tests DIR]... [--results junit.xml]
               [--root DIR] [--out DIR] [--judge none|mock|llm] [--strict]
               [--verbose [INFO|DEBUG]]
speccheck --self-check [--verbose [INFO|DEBUG]]
speccheck --version
speccheck --help
```

Subcommand / flag	Behavior	Exit
check	Run the §3.1 pipeline; write both reports; print the summary line.	0 conforming, 1 not, 2 usage, 3 input contract
--spec FILE	Required. Path to the specification; decoded as UTF-8 with errors="replace" (a Note is recorded if any byte was replaced). Missing flag or unreadable file → usage error.	2
--src DIR	Repeatable; default src if it exists, else none. Non-directory → usage error.	2

Subcommand / flag	Behavior	Exit
<code>--tests DIR</code>	Repeatable; default <code>tests</code> if it exists, else none. Non-directory → usage error.	2
<code>--results FILE</code>	Optional JUnit XML. Absent → every cited ID is at most UNVERIFIED. Unreadable → usage error; malformed → E-05.	2 / 3
<code>--root DIR</code>	Base for relative paths in reports; default <code>..</code> . All of <code>--spec/--src/--tests/--results/--out</code> MUST resolve inside it (E-09).	2
<code>--out DIR</code>	Where the two reports go; default <code>..</code> . MUST resolve inside <code>--root</code> (E-09; F-002). Created if missing; not writable → E-18.	2 / 3
<code>--judge MODE</code>	<code>none</code> (default), <code>mock</code> , <code>llm</code> . Any other value → usage error.	2
<code>--strict</code>	Apply R-15 and, with <code>--judge llm</code> , R-28.	—
<code>--max-unknown FRACTION</code>	Threshold for R-28; default 0.2; a float in <code>[0, 1]</code> , else usage error. Accepted with any judge mode; consulted only under <code>--strict --judge llm</code> .	2
<code>--judge-concurrency N</code>	K-06; default 4, integer 1..32, else usage error. Ignored unless <code>--judge llm</code> .	2
<code>--judge-budget SECONDS</code>	K-12; default 0 (unlimited), integer 0..86400, else usage error. Ignored unless <code>--judge llm</code> .	2
<code>--verbose [LEVEL]</code>	§5.3. Bare = INFO. LEVEL other than INFO/DEBUG → usage error.	2
<code>--self-check</code>	Copy the packaged fixture <code>speccheck/_selfcheck/</code> (a byte-identical copy of the §9.8 golden fixture <code>fixtures/target/</code> , its <code>golden/</code> directory included; F-107) into a fresh temporary directory <code><tmp></code> ; install a socket guard; then run, in the same process , exactly <code>check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --strict --root <tmp> --out <tmp>/out</code> with the working directory set to <code><tmp></code> (Q-003). The inner check is expected to exit 1 (the fixture has planted defects) and that exit code is not the self-check's result; compare <code><tmp>/out/speccheck.json</code> and <code><tmp>/out/SPEC_CONFORMANCE_REPORT.md</code> to <code><tmp>/golden/</code> byte-for-byte; remove <code><tmp></code> ; print <code>self-check: ok</code> when both match, else one line naming the first mismatch or failure (F-011).	0 / 1
<code>--version</code>	Print <code>speccheck <semver></code> and exit.	0

Summary line (stdout, exactly one line, only on exit 0/1):

```
speccheck: <STATUS> - <passing>/<in_scope> passing (<pct>%), <failing> failing, <skipped> skipped, <weal
```

The line above is shown unwrapped on purpose: it is one physical line, ASCII only, terminated by a single `\n`, and written to stdout as UTF-8 regardless of locale (R-29; F-010). `<STATUS>` is CONFORMING (exit 0) or NOT CONFORMING (exit 1); `<pct>` is conformance × 100 rounded half-even to one decimal; `<judge-suffix>` is empty unless R-28 forced exit 1, in which case it is (unavailable) or (unknown_rate <rate> > max_unknown <max>) with both numbers as in the JSON; when both R-28 reasons hold the suffix is (unavailable) (Q-004). The seven counts after the ratio are the seven in-scope statuses of C-07 by_status, each exactly once (Q-005).

Regex the line MUST match (T-44):

```
^speccheck: (CONFORMING|NOT CONFORMING) - \d+/\d+ passing \(\\d+\\.\\d%\\), \\d+ failing, \\d+ skipped, \\d+ w
```

5.2 GUI

None. This system has no graphical surface, and O-3 explicitly leaves one out of v0.1.

5.3 Diagnostics and verbosity contract

- **Quiet by default.** With no `--verbose`, the process writes nothing to stderr on exit 0/1 and only the usage/contract error message on exit 2/3.
- **Syntax.** `--verbose` alone $\equiv$ `--verbose INFO`; `--verbose INFO` and `--verbose DEBUG` are accepted; any other value is a usage error (exit 2).
- **INFO** (stderr): one line per pipeline stage with counts and elapsed milliseconds (files scanned, IDs declared, citations found, test cases attributed, results joined, edges judged), the judge mode, and for the LLM judge the URL and model. INFO MUST NOT include file contents, statement text, judge prompts, judge responses, or the API key.
- **DEBUG** (stderr): INFO plus, per judged edge, the full `JudgeRequest` JSON as sent and the raw provider response as received, each prefixed `judge>` / `judge<` on its own line. The API key is redacted. DEBUG MUST NOT change stdout or either report file.
- **Mechanism.** Python logging with a single stderr `StreamHandler`; logger name `speccheck`; format `%(levelname)s %(message)s`. Level is `ERROR` by default and `INFO`/`DEBUG` per flag, so nothing below `ERROR` reaches stderr without `--verbose` (F-014). The E-10..E-13 and E-27..E-30 Notes are logged at `INFO` (they are always in the report's Notes regardless of level); nothing is ever logged at `WARNING`; the exit-2/3 message is logged at `ERROR`.

5.4 Exit codes

Code	Meaning	Reports written?
0	CONFORMING: no in-scope ID is FAILING ; and if <code>--strict</code> , every in-scope ID has status PASSING , dangling and stale are empty, and — with <code>--judge llm</code> only — <code>judge_available</code> is true and <code>unknown_rate</code> $\leq$ <code>max_unknown</code> (R-28). Beyond R-28, status is the only judge-related input (F-003).	yes
1	NOT CONFORMING: anything else the report can express	yes
2	Usage error (bad flag/value, missing required, path outside root, missing judge env)	no
3	Input contract violation (E-01, E-02, E-03, E-05, E-18)	no

Any uncaught exception MUST also map to 3 with a one-line message; a traceback is printed only under `--verbose DEBUG`.

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	Read-only inputs. No file under <code>--root</code> other than the two report files under <code>--out</code> — and the <code>--out</code> directory itself, if it did not exist — is created, modified, or deleted by a run, and no temporary of §3.1 survives a run; no file outside <code>--root</code> is written at all. Sole exception: <code>--self-check</code> creates, uses, and removes one fresh temporary directory (F-011); it writes nowhere else.
I-002	Determinism. For <code>--judge none</code> and <code>--judge mock</code> , identical inputs (bytes of every file read, argv, env) produce byte-identical report files and summary line, on any OS.
I-003	Exactly-once reporting. Every declared ID (retired included) appears exactly once in <code>ids</code> and exactly once in the Markdown per-ID table; no undeclared ID ever appears there.

ID	Invariant
I-004	Downgrade-only judge. For every ID, <code>status_with_judge</code> $\in$ <code>{status_without_judge} $\cup$ ({WEAKLY_PASSING} iff <code>status_without_judge</code> == <code>PASSING</code>)</code> . Disabling the judge never changes any status except <code>WEAKLY_PASSING</code> $\rightarrow$ <code>PASSING</code> .
I-005	Grounded verdicts. Every non-UNKNOWN verdict in a report carries ≥ 0 evidence entries, every <code>ASSERTS</code> carries ≥ 1 , and every evidence entry points to a line inside the judged test case’s span in the judged file.
I-006	Network boundary. With <code>--judge none/mock</code> , no socket is opened for the lifetime of the process.
I-007	Secret and payload hygiene. The API key never appears in <code>stdout</code> , <code>stderr</code> , or either report. <code>INFO</code> never contains file contents, statements, prompts, or responses.
I-008	Total metrics. Every ratio is either a number in <code>[0, 1]</code> or <code>null/n/a</code> ; no run raises on a zero denominator.
I-009	Exit $\equiv$ report. <code>exit_code</code> in <code>speccheck.json</code> equals the process exit status, and both are computable from the rest of the JSON plus <code>strict</code> by the §5.4 rule.
I-010	One judge call per edge. The judge is invoked at most once per (test case, ID) edge per run, and only for edges whose ID is <code>PASSING</code> after C-05 step 4 and whose test outcome is <code>passed</code> .
I-011	Family-safe numbering. ID normalization is injective within a family: <code>R-7</code> , <code>[R-07] (#sec-contracts)</code> , <code>R-007</code> map to one ID; <code>[R-07] (#sec-contracts)</code> and <code>[C-07] (#C-07)</code> never collide.

7. Constraints (precise and measurable)

ID	Constraint
K-01	Usage errors exit 2; input-contract violations exit 3; conformance outcomes exit 0/1; no other exit codes exist.
K-02	Files larger than 2 MiB (2,097,152 bytes) under <code>--src/--tests</code> are skipped with a Note (E-10); binary files (a <code>0x00</code> byte within the first 8192 bytes) are skipped silently (E-29); text files that are not valid UTF-8 are decoded with <code>errors="replace"</code> and noted (E-11).
K-03	Directories named <code>.git</code> , <code>.hg</code> , <code>.svn</code> , <code>node_modules</code> , <code>__pycache__</code> , <code>.venv</code> , <code>venv</code> , and any directory whose name starts with <code>.</code> are never descended into; symbolic links to files or directories are never followed (E-30).
K-04	ID numbers are 1–3 digits; a token with 4+ digits is not an ID.
K-05	LLM judge timeout is <code>SPECHECK_JUDGE_TIMEOUT</code> seconds (default 30) per edge, wall-clock, measured from the moment the request is issued to the moment the full response body has been received, and covering every phase in between — DNS resolution, TCP connect, TLS handshake, request send, response receive (Q-010).
K-06	Exactly one HTTP request per judged edge; no retries, no batching. Requests MAY be issued concurrently, at most <code>--judge-concurrency</code> at a time (default 4, integer 1..32); output order is fixed by C-07 and MUST NOT depend on completion order (F-110).

ID	Constraint
K-12	<code>--judge-budget SECONDS</code> (default 0 = unlimited; integer 0..86400) bounds the wall-clock spent in the judge stage, measured from the first request issued. The deadline is <code>start + budget</code> . A request is <i>started</i> when it is issued; no request is issued at or after the deadline; requests in flight at the deadline are allowed to complete (each still bounded by K-05) and their verdicts count. Every edge not started before the deadline receives UNKNOWN with rationale <code>judge: budget</code> and <code>coerced: true</code> , and one Note records how many (F-110, Q-010).
K-07	<code>rationale</code> $\leq$ 280 characters after truncation; single line (newlines replaced by spaces).
K-08	Kernel performance, measured on the reference machine named in <code>SPEC_BUILD_REPORT.md</code> (CPU model, RAM, OS, Python build, run in isolation): <code>check --judge mock</code> on the §9.8 golden fixture completes in ≤ 2 s wall-clock, and a generated 10,000-file tree with 100 declared IDs in ≤ 60 s. Both bounds are recorded, not CI-gated (F-017).
K-09	JSON output is UTF-8 , <code>indent=2</code> , <code>ensure_ascii=False</code> , sorted per C-07 (not alphabetically), trailing newline; Markdown output is UTF-8 with <code>\n</code> line endings.
K-10	<code>--version</code> prints a PEP 440 version equal to the package metadata version.
K-11	<code>--max-unknown</code> defaults to 0.2, accepts a decimal in [0, 1] (parsed as a Decimal from its literal text), and is compared to the quantized <code>unknown_rate</code> of C-07 (Decimal, 4 places, <code>ROUND_HALF_EVEN</code> ; Q-009); equality passes; a <code>null unknown_rate</code> (zero judged edges) never exceeds it (F-105).

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Spec declares zero IDs (or all declared IDs are retired)	Exit 3, message <code>spec declares no in-scope IDs</code> ; no reports written.
E-02	Same ID declared twice (both non-retired, or both retired)	Exit 3, message names the ID and both lines.
E-03	ID declared retired and also declared non-retired (any order)	Exit 3, message names the ID and both lines (a spec must retire <i>or</i> keep an ID, not both).
E-04	ID declared inside a fenced code block in <code>SPEC.md</code>	Not a declaration; ignored silently (spec templates contain example IDs).
E-05	<code>--results</code> file is not well-formed XML, or root is neither <code>testsuites</code> nor <code>testsuite</code> , or a <code><testcase></code> lacks <code>name</code>	Exit 3, message <code>results: <reason></code> .
E-06	Duplicate (<code>classname</code> , <code>name</code>) in results	Single outcome = worst of them (C-04); a Note records the count.
E-07	Result <code><testcase></code> joins no attributed test case	Listed under <code>unattributed_results</code> ; not counted for any ID; exit unaffected.
E-08	Test case cites an ID but has no result (results absent, or case not run)	Case listed under that ID's <code>unrun</code> ; ignored in C-05 step 4; if <i>no</i> citing case has a result $\rightarrow$ UNVERIFIED.

ID	Case	Semantics
E-09	Any of <code>--spec/--src/--tests/--results</code> and <code>--root</code> resolves outside <code>--root</code> (after symlink resolution)	Exit 2, message <code>path outside --root: <path></code> . A <code>results/</code> inside <code>--root</code> is used in resolved form and descended; it is not an E-30 skip (Q-007).
E-10	File > 2 MiB under a scan root	Skipped; Note <code>skipped 1 file over 2 MiB: <path></code> .
E-11	A scanned file — or <code>SPEC.md</code> itself — is not valid UTF-8	Decoded with replacement; Note; citations on replaced bytes are impossible (token requires ASCII) so no false citations arise.
E-12	<code>.py</code> test file fails to parse	Whole file is one file-level case; Note <code>parse fallback: <path></code> .
E-13	Test citation lands in a file-level case (module docstring, helper, fixture)	Attributed to the file-level case; that case has no JUnit outcome by construction (name ""), so it can only ever contribute <code>UNVERIFIED</code> ; the JSON carries <code>"name": ""</code> and the Markdown renders the label (file) (Q-011).
E-14	Judge provider unreachable, times out, or returns <code>HTTP ≥ 400</code>	Verdict <code>UNKNOWN</code> , <code>coerced: true</code> , rationale <code>judge: unavailable / judge: timeout / judge: http <code></code> ; <code>judge_available</code> set false if <i>every</i> call failed; exit unaffected unless <code>--strict</code> (E-32).
E-15	Judge returns non-JSON or JSON lacking verdict	<code>UNKNOWN</code> , <code>coerced: true</code> , rationale <code>judge: malformed response</code> .
E-16	Judge returns <code>ASSERTS</code> with no evidence, or evidence outside the span / in another file	<code>UNKNOWN</code> , <code>coerced: true</code> , rationale <code>judge: ungrounded</code> . The raw answer is available only at <code>DEBUG</code> .
E-17	Judge returns a valid verdict for an ID whose test span contains no code at all (empty function)	Verdict recorded as returned; the kernel does not second-guess content, only grounding.
E-18	<code>--out</code> cannot be created, or a temporary or final report file cannot be written or renamed	Exit 3; message <code>out: <reason></code> ; on exit neither final report from this run exists and no <code>.tmp</code> from this run remains. Because renames happen JSON-then-Markdown (§3.1), the only stale combination an operator can find is: a previous run's <code>SPEC_CONFORMANCE_REPORT.md</code> with no <code>speccheck.json</code> (F-106).
E-19	Both <code>--src</code> and <code>--tests</code> empty/absent	Run proceeds; every in-scope ID is <code>UNCITED</code> ; exit 1 (or 0 only if <code>in_scope == 0</code> , which is E-01).
E-20	Dangling citation of a token that <i>looks</i> retired in code (<code>~~R-99~~</code> in a comment)	Strikethrough is only meaningful in <code>SPEC.md</code> ; in code it is a plain citation of <code>R-99</code> → dangling.
E-21	<code>--judge llm</code> with a required env var missing	Exit 2, message names the variable, key value never echoed.
E-22	Test file cites the same ID on several lines of one case	One edge; <code>lines</code> lists every line.
E-23	<code>--spec</code> , <code>--results</code> , or a report path lies under a <code>--src/--tests</code> root (e.g. <code>--src .</code> , or <code>--out</code> inside <code>src/</code>)	Those files are not scanned and produce no citations; no Note; a second run over the same inputs is byte-identical to the first (F-002).

ID	Case	Semantics
E-24	JUnit name carries a parametrization suffix (<code>test_x[3-True]</code>), or several <code><testcase></code> join one case	Suffix stripped for the join; each raw result kept in results ; the case's outcome is the worst of them (F-005).
E-25	A T id is cited in source files but by no test case	UNCITED (source citations are evidence only for family T); listed under src in the report (F-001).
E-26	An ID has judge verdicts of both ASSERTS and EXECUTES_ONLY on passed edges	Status PASSING ; passes <code>--strict</code> ; the EXECUTES_ONLY edge remains visible in §8 of the report (F-003).
E-27	A result's classname suffix-matches two test cases equally (<code>tests/a/test_core.py</code> and <code>tests/b/test_core.py</code> , result classname <code>test_core</code>)	Unattributed; one Note names the result and all candidates; neither case receives the outcome (F-006).
E-28	A <code>test*</code> function (underscore or not) is a method of a class that is neither <code>Test*</code> -named nor a <code>*TestCase</code> subclass, or of a nested class	Not a test case; its citations are file-level; one Note per file lists the undelimited names (F-007, F-102).
E-29	A file under a scan root contains a <code>0x00</code> byte in its first 8192 bytes	Skipped silently: no citations, no test case, no Note (F-008).
E-30	A symbolic link (file or directory) is encountered <i>during descent</i> under a scan root	Not followed; one Note per run: skipped N symlink(s) (F-008). A symlink that is itself a command-line root is E-09 's case, not this one (Q-007).
E-31	A bold ID token appears in a table cell other than the first, or in a heading after other tokens	Not a declaration (F-009). It is also not a citation: <code>SPEC.md</code> is never scanned for citations.
E-32	<code>--strict --judge llm</code> and either <code>judge_available == false</code> or <code>unknown_rate > max_unknown</code>	Exit 1 even if every status is PASSING ; <code>strict_judge_failure</code> set to "unavailable" if the judge was unavailable (whether or not the rate also exceeds), else "unknown_rate"; summary line carries the matching suffix (F-012, Q-004). Without <code>--strict</code> , or with <code>--judge none mock</code> , no effect.
E-33	A line contains <code>speccheck:ignore</code> , or a file's first three lines contain <code>speccheck:ignore-file</code>	The line yields no citations / the file is not scanned; ignored files are counted in one Note (F-013).

ID	Case	Semantics
E-34	A .speccheck.json.*.tmp or .SPEC_CONFORMANCE_REPORT.md.*.tmp from a killed run exists under --out, which lies under a scan root	Never scanned (C-03); deleted at §3.1 step 1; the run's reports are byte-identical to a run without the leftover (F-103).
E-35	--judge llm and the --judge-budget deadline passes with edges not yet started	Each such edge is UNKNOWN, coerced: true, rationale judge: budget; in-flight requests complete and count; one Note judge budget exhausted: N edge(s) unjudged; unknown_rate counts them; R-28 applies as usual (F-110, Q-010).
E-36	--judge llm with zero eligible edges	No request is sent; judge_available is true; unknown_rate is null and never trips R-28 (F-105).
E-37	A tests[] entry whose case was not judged: judge disabled, or the ID not PASSING after C-05 step 4, or the outcome not passed	JSON verdict is null (key present); Markdown renders an em dash in the verdict position; the entry never appears in report §8 (Q-001).
E-38	Two or more Notes are produced in one run	Emitted in ascending Unicode code-point order of their full text, in both reports (Q-002).

9. Acceptance criteria, tests, and evals

Every test cites, in its docstring or a comment, its own T id and the R/C/I/K/E ids it proves, so that speccheck can check itself (§9.9; F-001). Test inputs that contain the literal marker strings of [C-01](#) (speccheck:ignore, speccheck:ignore-file) MUST live in data files under tests/data/, never inline in a test module, so self-application cannot ignore its own tests (F-109). Groups are ordered by pipeline stage.

9.1 Extraction ([C-01](#), [C-02](#))

ID	Test
T-01	Table-cell and heading declarations are both extracted with correct family, number, statement, and line; a SPEC.md containing invalid UTF-8 is decoded with replacement, its ASCII declarations are still found, and a Note is recorded. (R-01 , E-11)
T-02	R-7, [R-07] (#sec-contracts), R-007 in a spec are one ID reported as [R-07] (#sec-contracts); I-5 is reported as [I-005] (#sec-invariants). (I-011 , K-04)
T-03	A token with 4 digits (R-1234) and tokens adjacent to alphanumerics (XR-07, R-07a) are not IDs. (K-04 , C-01)
T-04	Strikethrough declarations yield retired=True; a later plain declaration of the same ID exits 3. (R-02 , E-03)

ID	Test
T-05	IDs inside fenced code blocks in <code>SPEC.md</code> are ignored, for both <code>```</code> and <code>~~~</code> fences, with and without an info string, and for an unclosed fence running to end of file; a backtick fence opened with an info string is closed by a bare backtick line, is not closed by a tilde line, and a candidate closing line carrying an info string does not close it. (E-04 , C-01)
T-06	Duplicate declaration exits 3 and names both lines. (E-02)
T-55	A bold ID in a second cell, a heading with the ID after other tokens, a separator row, and a row with <code>\ </code> and a backtick-quoted pipe in a cell are parsed per C-01 : only first-cell/first-token forms declare; the statement is the second cell with escapes intact. (E-31 , C-01)
T-07	A spec with no in-scope IDs exits 3 and writes no files. (E-01 , I-001)

9.2 Citation and attribution (C-03)

ID	Test
T-08	Source citations are recorded per (file, line) for tokens in code, comments, and strings alike. (R-03)
T-09	Python test citations inside <code>def test_*</code> and <code>class Test*</code> methods are attributed to the enclosing case with the correct <code>classname</code> , <code>start</code> , <code>end</code> . (R-04)
T-10	A citation in a module docstring or helper is attributed to the file-level case and reported with name (<code>file</code>). (E-13)
T-11	An unparseable <code>.py</code> test file falls back to one file-level case and produces a Note. (E-12)
T-12	A non-Python test file (<code>.go</code> , <code>.ts</code>) gets file-level attribution. (R-04 , O-2)
T-13	Excluded directories (K-03), oversized / non- UTF-8 files (K-02), binary files, and symlinks are skipped or decoded exactly as specified: oversized and non- UTF-8 produce Notes, binary produces none, symlinks produce one Note per run and are never followed (a symlink cycle terminates). (E-10 , E-11 , E-29 , E-30)
T-56	<code>test_*</code> and <code>testFoo</code> methods of a <code>*TestCase</code> subclass, <code>testFoo</code> methods of a <code>Test*</code> class, and <code>async def test_*</code> functions are delimited with correct spans; a module-level <code>testFoo</code> (no underscore) is not a test case; a <code>test*</code> method of an unrecognized class is not, its citations are file-level, and the file gets an <code>undelimited tests</code> Note. (E-28 , C-03)
T-57	A line containing <code>speccheck:ignore</code> yields no citations but still counts toward its case's span; a file with <code>speccheck:ignore-file</code> on line 2 yields no citations and no test cases; a marker on line 4 has no file-level effect; markers inside <code>SPEC.md</code> change nothing; the Note counts ignored files. (R-27 , E-33)
T-14	Several citations of one ID in one case yield one edge with all lines listed. (E-22)

9.3 Results mapping (C-04)

ID	Test
T-15	<code><testsuites></code> and bare <code><testsuite></code> roots both parse; <code>failure/error/skipped</code> children map to the correct outcome. (R-05)
T-16	Classname join accepts both <code>tests.test_core</code> and <code>test_core</code> forms for <code>tests/test_core.py</code> , and rejects <code>unit_test_core</code> for <code>test_core</code> . (C-04)
T-58	With <code>tests/a/test_core.py</code> and <code>tests/b/test_core.py</code> , a result with classname <code>tests.a.test_core</code> joins <code>a</code> only; a result with classname <code>test_core</code> is unattributed with a Note naming both candidates. (E-27 , C-04)
T-52	<code>test_x[3-True]</code> and <code>test_x[0-False]</code> both join <code>test_x</code> ; <code>results</code> lists both with <code>param</code> ; the case <code>outcome</code> is the worst of them; a name with no <code>[]</code> is unchanged; everything from the first <code>[]</code> is removed: <code>test_y[a][b] → test_y</code> and <code>test_x[list[int]] → test_x</code> with <code>param list[int]</code> ; a result with empty classname joins the unique case with its <code>join_name</code> and is unattributed when two exist. (E-24 , C-04 , F-105)
T-17	Duplicate <code>(classname, name)</code> collapses to the worst outcome in the order <code>error > failed > skipped > passed</code> . (E-06)
T-18	Results for unknown cases are listed as unattributed and change no status. (E-07)
T-19	Malformed XML and a <code><testcase></code> without <code>name</code> each exit 3. (E-05)

9.4 Status algorithm and metrics ([C-05](#), [C-07](#))

ID	Test
T-20	One fixture per deterministic status — <code>UNCITED</code> , <code>UNTESTED</code> , <code>UNVERIFIED</code> , <code>FAILING</code> , <code>SKIPPED</code> , <code>PASSING</code> , <code>RETIRED</code> — each asserting exactly that status; the <code>UNCITED</code> fixture has empty <code>--src</code> and <code>--tests</code> and exits 1. (R-06 , E-19)
T-21	An ID with one failed and three passed citing tests is <code>FAILING</code> ; with all citing tests skipped is <code>SKIPPED</code> ; with some skipped and some passed is <code>PASSING</code> . (C-05 step 4)
T-22	Citing cases absent from results are listed under <code>unrun</code> and do not affect the status. (E-08)
T-23	Dangling and stale citations are listed with file and line; strikethrough in code does not retire anything. (R-07 , R-08 , E-20)
T-24	<code>conformance</code> , per-family ratios, and counts match hand-computed values on the golden fixture; a family with zero in-scope IDs reports <code>null/n/a</code> , not <code>0.0</code> , and nothing raises. (R-09 , I-008)
T-25	Retired IDs are excluded from every denominator and still appear in <code>ids</code> exactly once. (R-02 , I-003)
T-53	A T id cited only in a source file is <code>UNCITED</code> ; a T id cited by a passing test is <code>PASSING</code> ; T ids are counted in <code>in_scope</code> , <code>conformance</code> , and <code>by_family.T</code> . (R-25 , E-25)

9.5 Judge contract ([C-06](#))

ID	Test
T-26	Mock judge returns <code>ASSERTS</code> with one evidence line per assertion token, and <code>EXECUTES_ONLY</code> with no evidence when none is present, including for an empty test body. (R-22 , E-17)
T-27	A <code>PASSING</code> ID whose only passed edges are <code>EXECUTES_ONLY</code> becomes <code>WEAKLY_PASSING</code> ; with any <code>ASSERTS</code> edge it stays <code>PASSING</code> and passes <code>--strict</code> ; with only <code>UNKNOWN</code> it stays <code>PASSING</code> . (C-05 step 5, R-11 , E-26)
T-28	Disabling the judge changes no status except <code>WEAKLY_PASSING</code> → <code>PASSING</code> (property test over random fixtures). (I-004)
T-29	A stub provider returning <code>ASSERTS</code> without evidence, evidence outside the span, or evidence in another file is coerced to <code>UNKNOWN</code> with <code>coerced: true</code> and rationale <code>judge: ungrounded</code> . (E-16 , I-005)
T-30	Non-JSON, missing <code>verdict</code> , an unknown verdict string, a raised exception, and a timeout each yield <code>UNKNOWN</code> with the specified rationale. (E-14 , E-15)
T-31	The judge is called exactly once per eligible edge and never for <code>FAILING/SKIPPED/UNVERIFIED/UNTESTED/UNCITED</code> IDs or for skipped/failed test outcomes (call-counting stub). (I-010)
T-32	Rationale over 280 chars is truncated to 277 + ...; newlines become spaces. (K-07)
T-33	LLM provider sends exactly one <code>POST</code> per edge whose body is byte-for-byte the C-06 shape (<code>model</code> , <code>temperature: 0</code> , <code>max_tokens: 4000</code> , system message equal to the C-10 text, user message equal to the request JSON with line-numbered <code>source</code>), with the bearer header, honors the timeout, never retries, and issues at most <code>--judge-concurrency</code> requests at once while output order stays fixed (recorded HTTP stub with a concurrency counter). (K-05 , K-06 , R-26)
T-54	The provider reads <code>choices[0].message.content</code> , accepts a bare JSON object and one wrapped in a <code>```/```json</code> fence, and treats a missing path, a non-200 status, or any other text as E-14/E-15 ; the report carries <code>judge_prompt_sha256</code> equal to the SHA-256 of <code>speccheck/judge_prompt.md</code> , and that file equals the C-10 text. (C-06 , C-10 , R-26)

9.6 Reports and determinism ([C-07](#), [C-08](#))

ID	Test
T-34	<code>speccheck.json</code> validates against the C-07 shape: key order, sort orders, rounding, no absolute paths, no timestamps; <code>by_family</code> has all six keys and <code>by_status</code> all seven in-scope statuses on every fixture including single-family ones; <code>judge_available</code> is <code>null</code> for <code>none</code> , <code>true</code> for <code>mock</code> , <code>true</code> for <code>llm</code> with zero eligible edges, <code>false</code> for <code>llm</code> when every call fails; every <code>tests[]</code> entry carries the <code>verdict</code> key, <code>null</code> on a <code>FAILING</code> ID, on a skipped-outcome edge of a <code>PASSING</code> ID, and everywhere under <code>--judge none</code> ; <code>tests[].name</code> is <code>"</code> for a file-level case; a fixture producing five Notes emits them in code-point order in both reports; every ratio has exactly four decimal places and a hand-picked tie value (e.g. $1/8 = 0.125$ at 3 places, 0.00005 at 4) rounds half-even to the quantized Decimal, not to <code>round()</code> 's binary result. (R-13 , R-20 , K-09 , C-07 , E-37 , E-38)
T-35	<code>SPEC_CONFORMANCE_REPORT.md</code> has the nine C-08 sections in order, one per-ID row per declared ID, retired rows with exactly the ID cell struck through, an em dash in the verdict position for every unjudged case and <code>(file)</code> for every file-level case, and every <code>file:line</code> in it also present in the JSON. (R-12 , I-003 , E-37 , Q-011)
T-36	Two runs on the golden fixture with <code>--judge mock</code> produce byte-identical reports and summary lines; the same holds after copying the fixture to a different absolute path, and with <code>--src .</code> and <code>--out</code> inside the source root (the previous run's reports and the spec itself produce no citations), and with a planted <code>.speccheck.json.deadbeef.tmp</code> and <code>.SPEC_CONFORMANCE_REPORT.md.deadbeef.tmp</code> under <code>--out</code> before the run (never scanned, deleted by the run). (R-16 , I-002 , E-23 , E-34)
T-37	Every metric in the report is recomputable from the report's own evidence table plus the results file (the test recomputes them independently). (R-24)
T-38	Only the two report files are created; input trees are byte-identical before and after (hash comparison). (R-19 , I-001)

9.7 CLI, exit codes, diagnostics, boundary (§5)

ID	Test
T-39	<code>exit_code</code> in JSON equals the process exit status for fixtures exercising 0, 1 (non-strict), 1 (strict-only reasons: <code>UNVERIFIED</code> , <code>UNTESTED</code> , <code>UNCITED</code> , <code>SKIPPED</code> , <code>WEAKLY_PASSING</code> , <code>dangling</code> , <code>stale</code>), and 0 under <code>--strict</code> for a mixed <code>ASSERTS/EXECUTES_ONLY</code> ID. (R-14 , R-15 , I-009 , E-26)
T-40	Missing <code>--spec</code> , bad <code>--judge</code> , bad <code>--verbose</code> value, any input or <code>--out</code> path outside <code>--root</code> , and missing <code>judge</code> env each exit 2 with the specified message; the API key value never appears in the message. (K-01 , E-09 , E-21 , R-23)
T-41	Bare <code>--verbose</code> $\equiv$ <code>--verbose INFO</code> ; <code>INFO</code> emits stage lines with counts and no statement text, file contents, or prompts; <code>DEBUG</code> emits <code>judge>/judge<</code> lines with the key redacted; <code>stdout</code> is the single summary line in all cases. (R-17 , I-007)
T-42	With no <code>--verbose</code> , <code>stderr</code> is empty on exit 0/1 even when the run produced Notes; with <code>--verbose</code> , each Note appears once at <code>INFO</code> . (§5.3)

ID	Test
T-43	With <code>--judge none</code> and <code>--judge mock</code> , a socket-creation guard installed for the process lifetime is never triggered; <code>speccheck --self-check</code> passes from an installed wheel in an empty working directory, invokes the inner check in-process with exactly the §5.1 argument list (asserted via a recorded <code>Config</code>), prints <code>self-check: ok</code> although its inner check exits 1, leaves no files behind, and writes nothing outside its temporary directory. (R-18 , I-006 , I-001 , Q-003)
T-60	<code>speccheck/_selfcheck/</code> is byte-identical to <code>fixtures/target/</code> (recursive file-by-file comparison, <code>golden/</code> included); the test fails if either side changes without the other. (F-107 , §10)
T-44	Summary line matches the §5.1 regex exactly, is pure ASCII, ends in a single <code>\n</code> , is emitted correctly under a C/POSIX locale and a <code>cp1252</code> stdout, carries all seven in-scope status counts including <code>skipped</code> , and its numbers equal the JSON metrics. (R-21 , R-29 , Q-005)
T-59	Under <code>--strict --judge llm</code> with a provider stub that fails every call — so that <code>judge_available</code> is <code>false</code> and <code>unknown_rate</code> is <code>1.0000 > max_unknown</code> — exit is 1, <code>strict_judge_failure</code> is <code>"unavailable"</code> (not <code>"unknown_rate"</code>), and the summary line ends <code>judge=llm (unavailable)</code> (Q-004); with a stub yielding 3 UNKNOWN of 10 edges and <code>--max-unknown 0.2</code> , exit is 1 with <code>"unknown_rate"</code> and the numeric suffix; with <code>--max-unknown 0.3</code> exit is 0; without <code>--strict</code> neither affects exit; with a fixture that has zero eligible edges, no request is sent, <code>unknown_rate</code> is <code>null</code> , and <code>--strict</code> exits 0. (R-28 , K-11 , E-32 , E-36)
T-61	With <code>--judge-budget 1</code> , <code>--judge-concurrency 2</code> , and a provider stub that sleeps 2 s per call over six edges, exactly the two requests issued before the deadline complete and are judged (in-flight requests are not cancelled); the remaining four are UNKNOWN with rationale <code>judge: budget</code> and <code>coerced: true</code> ; the Note reports 4; <code>unknown_rate</code> includes them; with <code>--judge-budget 0</code> all six are judged. (K-12 , E-35 , Q-010)
T-45	Unwritable <code>--out</code> exits 3 with nothing written; a failure injected after the JSON rename and before the Markdown rename exits 3 with no <code>.tmp</code> left, no <code>speccheck.json</code> , and the previous run's <code>SPEC_CONFORMANCE_REPORT.md</code> intact (the one stale combination E-18 permits); leftover temporaries from an earlier run are deleted first; on success no <code>.tmp</code> remains and the two temporaries used carried the same 8-hex-digit nonce. (E-18 , I-001 , §3.1)
T-50	<code>speccheck --version</code> prints <code>speccheck <version></code> where <code><version></code> is PEP 440 and equals <code>importlib.metadata.version("speccheck")</code> . (K-10)

9.8 Golden fixture (end to end)

ID	Test
T-46	<code>fixtures/target/</code> — a small self-contained project with its own <code>SPEC.md</code> (≥ 12 IDs across all six families, 2 retired), <code>src/</code> , <code>tests/</code> , and a checked-in <code>junit.xml</code> — contains planted defects: one UNCITED R, one UNTESTED C, one UNVERIFIED E, one FAILING T, one SKIPPED K, one EXECUTES_ONLY-only test, one dangling citation, one stale citation, one unattributed result, one file-level citation. The golden reports live in <code>fixtures/target/golden/</code> (outside every scan root). <code>speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --strict --root fixtures/target --out <fresh tmp></code> produces files byte-identical to <code>golden/speccheck.json</code> and <code>golden/SPEC_CONFORMANCE_REPORT.md</code> and exits 1 (Q-003). (all of §2)
T-47	Removing each planted defect in turn flips exactly the expected row and metric (one sub-test per defect). (R-06 , R-24)

9.9 Self-application (recorded, not gating)

ID	Test
T-48	<code>speccheck check --spec SPEC.md --src src --tests tests --results <this suite's junit.xml> --judge mock</code> on this repository reports every R/C/I/K/E/T ID in this document as PASSING. Recorded in <code>SPEC_BUILD_REPORT.md</code> ; it is evidence <i>for</i> the build, not the proof of it — the proof is §9.1–§9.8. (R-24)

9.10 Performance (recorded)

ID	Test
T-51	A benchmark script (<code>tools/bench.py</code> , not collected by pytest) runs <code>check --judge mock</code> on the golden fixture and on the generated 10,000-file tree five times each and prints the median wall-clock; the medians, the reference-machine description, and the pass/fail against K-08 are recorded in <code>SPEC_BUILD_REPORT.md</code> . Not run in CI. (K-08)

9.11 LLM judge evaluation (opt-in, --judge llm, not run in CI)

ID	Test
T-49	On the golden fixture’s judged edges (each hand-labeled ASSERTS/EXECUTES_ONLY/UNRELATED), the LLM judge’s non-UNKNOWN verdicts agree with labels at ≥ 0.90 accuracy and <code>unknown_rate</code> ≤ 0.10 in each of three independent runs (no pooling; one failing run fails T-49); all three per-run figures are recorded with model name, date, and <code>judge_prompt_sha256</code> . (R-10 , R-26)

10. Dependencies and environment

- **Runtime:** Python 3.12; uv for environment and lockfile. Install: `uv sync --extra dev`; with the model judge: `uv sync --extra dev --extra llm`.
- **Kernel dependencies:** none beyond the standard library (`re`, `ast`, `xml.etree.ElementTree`, `json`, `argparse`, `pathlib`, `logging`, `hashlib`).
- **Package data:** `speccheck/judge_prompt.md` (C-10) and `speccheck/_selfcheck/` (a byte-identical copy of `fixtures/target/` including its `golden/` directory, kept in sync by T-60; F-011, F-107), both included in the wheel.
- **Golden fixture layout:** `fixtures/target/{SPEC.md, src/, tests/, junit.xml, golden/speccheck.json, golden/SPEC_CONFORMANCE_REPORT.md}`; `golden/` is never a scan root and is never written to by a run (Q-003).
- **[llm] extra:** `httpx` (HTTP client with per-request timeout). No provider SDK; the wire format is C-06.
- **[dev] extra:** `pytest`, `pytest-cov`, `hypothesis` (T-28), `ruff`.
- **Environment variables:** C-09 (only with `--judge llm`). No configuration files are read.
- **Host prerequisites:** none. No network access is required except with `--judge llm`.
- **Commands:**

```
uv sync --extra dev
uv run python -m pytest tests -q --junitxml=junit.xml      # the $9 suite; junit.xml feeds T-48
uv run ruff check src tests
uv run speccheck --self-check
uv run speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock
```

- **Optional items:**
 - **O-1** LLM judge provider (`--judge llm`, [llm] extra). Specified fully in C-06/C-09; gated by flag and extra.
 - **O-2** Additional language adapters for test-case attribution (Go `func Test*`, JS/TS `test(/it()`). Not in v0.1; the fallback path is what v0.1 ships and tests (T-12).
 - **O-3** Any non-CLI surface (GUI, HTTP, editor plugin). Not in v0.1 and not designed for.

11. Traceability matrix (id → where realized)

Until the build exists, “where realized” names the component the §1/§4 design assigns; `spec-build` replaces it with the real module and the real test.

Spec id	Where realized (component / module)	Verified by
R-01	<code>extract.py</code> (declaration scan, UTF-8 replace)	T-01, T-05
R-02	<code>extract.py</code> (retired flag), <code>graph.py</code> (denominators)	T-04, T-25
R-03	<code>extract.py</code> (source scan, C-03 exclusions, binary/symlink filters)	T-08, T-13, T-36
R-04	<code>attribute.py</code> (Python adapter, fallback, C-03 exclusions)	T-09, T-10, T-11, T-12, T-36, T-56
R-05	<code>results.py</code>	T-15, T-16, T-52, T-58
R-06	<code>graph.py</code> (C-05 steps 1–4)	T-20, T-21, T-47
R-07	<code>graph.py</code> (dangling)	T-23
R-08	<code>graph.py</code> (stale)	T-23
R-09	<code>graph.py</code> (metrics)	T-24
R-10	<code>judge.py</code> , <code>judge_mock.py</code> , <code>judge_llm.py</code> (passed-outcome edges only)	T-26, T-31, T-33, T-49
R-11	<code>graph.py</code> (C-05 step 5)	T-27, T-28
R-12	<code>report.py</code> (Markdown)	T-35
R-13	<code>report.py</code> (JSON)	T-34

Spec id	Where realized (component / module)	Verified by
R-14	cli.py (exit mapping)	T-39
R-15	cli.py (--strict, status-only)	T-39, T-27
R-16	all kernel modules (sorted, no volatile fields)	T-36
R-17	cli.py (logging setup, level ERROR default)	T-41, T-42
R-18	cli.py (--self-check, pinned in-process invocation), speccheck/_selfcheck/, judge_llm.py import isolation	T-43
R-19	report.py (only writer, temp-and-rename), cli.py	T-38, T-43, T-45
R-20	extract.py, report.py (path normalization)	T-34, T-36
R-21	cli.py (summary line)	T-44, T-59
R-22	judge_mock.py	T-26
R-23	judge_llm.py, cli.py (env validation, redaction)	T-40, T-41
R-24	report.py (evidence table completeness)	T-37, T-48
R-25	graph.py (C-05 step 2b)	T-53
R-26	judge_llm.py (body, response path, prompt hash), speccheck/judge_prompt.md	T-33, T-54
R-27	extract.py (ignore markers)	T-57
R-28	cli.py (--strict judge gate), report.py (strict_judge_failure)	T-59
R-29	cli.py (summary line encoding)	T-44
C-01	extract.py (ID_RE, fence tracker, row/heading parsers, ignore markers)	T-01, T-02, T-03, T-04, T-05, T-55, T-57
C-02	extract.py (SpecId, SpecIndex)	T-01, T-06
C-03	attribute.py (TestCase, Citation, test* methods), extract.py (exclusions incl. temporaries, binary, symlinks)	T-09, T-10, T-13, T-14, T-36, T-56
C-04	results.py	T-15, T-16, T-17, T-18, T-19, T-52, T-58
C-05	graph.py (IdStatus, compute_status)	T-20, T-21, T-27, T-53
C-06	judge.py (JudgeRequest with numbered source, Verdict, validation), providers	T-26, T-29, T-30, T-32, T-33, T-54
C-07	report.py (to_json; Decimal quantization; verdict: null; Note order)	T-34, T-37, T-59
C-08	report.py (to_markdown; em dash and (file) renderings)	T-35
C-09	judge_llm.py (from_env)	T-33, T-40
C-10	speccheck/judge_prompt.md, judge_llm.py (system message)	T-33, T-54
I-001	report.py (temp-and-rename), cli.py	T-07, T-38, T-43, T-45
I-002	kernel modules	T-36
I-003	report.py	T-25, T-35
I-004	graph.py	T-27, T-28
I-005	judge.py (validation)	T-29
I-006	cli.py, module import layout	T-43
I-007	cli.py (logging filter), judge_llm.py (redaction)	T-40, T-41
I-008	graph.py (metrics)	T-24
I-009	cli.py, report.py	T-39
I-010	graph.py (edge selection), judge.py	T-31
I-011	extract.py (normalization)	T-02
K-01	cli.py	T-39, T-40, T-19
K-02	extract.py (file filters, binary rule)	T-13
K-03	extract.py (walker, no symlinks)	T-13

Spec id	Where realized (component / module)	Verified by
K-04	<code>extract.py</code> (ID_RE)	T-03
K-05	<code>judge_llm.py</code> (full-lifecycle timeout)	T-33
K-06	<code>judge_llm.py</code> (concurrency), <code>cli.py</code> (<code>--judge-concurrency</code>)	T-33
K-07	<code>judge.py</code>	T-32
K-08	kernel (performance)	T-51
K-09	<code>report.py</code>	T-34
K-10	<code>cli.py</code> (<code>--version</code>)	T-50
K-11	<code>cli.py</code> (<code>--max-unknown</code> , Decimal comparison, null rule)	T-59
K-12	<code>judge.py</code> (budget deadline, in-flight completion), <code>cli.py</code> (<code>--judge-budget</code>)	T-61
E-01	<code>extract.py</code> , <code>cli.py</code>	T-07
E-02	<code>extract.py</code>	T-06
E-03	<code>extract.py</code>	T-04
E-04	<code>extract.py</code> (fence tracking)	T-05
E-05	<code>results.py</code>	T-19
E-06	<code>results.py</code>	T-17
E-07	<code>results.py</code> , <code>report.py</code>	T-18
E-08	<code>graph.py</code> (unrun)	T-22
E-09	<code>cli.py</code> (path validation incl. <code>--out</code> ; symlinked roots resolved and used)	T-40
E-10	<code>extract.py</code>	T-13
E-11	<code>extract.py</code> (scan roots and SPEC.md)	T-13, T-01
E-12	<code>attribute.py</code>	T-11
E-13	<code>attribute.py</code> , <code>report.py</code>	T-10
E-14	<code>judge.py</code> , <code>judge_llm.py</code>	T-30, T-33
E-15	<code>judge.py</code>	T-30
E-16	<code>judge.py</code>	T-29
E-17	<code>judge.py</code>	T-26
E-18	<code>report.py</code> (write/rename/cleanup, JSON-then-Markdown)	T-45
E-19	<code>graph.py</code> , <code>cli.py</code>	T-20, T-39
E-20	<code>extract.py</code> (strikethrough only in spec)	T-23
E-21	<code>cli.py</code>	T-40
E-22	<code>attribute.py</code>	T-14
E-23	<code>extract.py</code> (exclusions)	T-36
E-24	<code>results.py</code> (join_name, worst-of)	T-52
E-25	<code>graph.py</code> (step 2b)	T-53
E-26	<code>graph.py</code> (step 5), <code>cli.py</code> (<code>--strict</code>)	T-27, T-39
E-27	<code>results.py</code> (longest-suffix join)	T-58
E-28	<code>attribute.py</code> (class recognition, <code>test*</code> methods)	T-56
E-29	<code>extract.py</code> (binary rule)	T-13
E-30	<code>extract.py</code> (walker; descent-time symlinks only)	T-13
E-31	<code>extract.py</code> (row/heading parsers)	T-55
E-32	<code>cli.py</code> (R-28 gate, <code>unavailable</code> precedence)	T-59
E-33	<code>extract.py</code> (ignore markers)	T-57
E-34	<code>extract.py</code> (temporary exclusion), <code>report.py</code> (leftover cleanup)	T-36, T-45
E-35	<code>judge.py</code> (budget)	T-61
E-36	<code>judge.py</code> , <code>cli.py</code> (R-28 with null)	T-59
E-37	<code>report.py</code> (<code>verdict: null</code> , em-dash rendering)	T-34, T-35
E-38	<code>report.py</code> (Note ordering)	T-34

12. Open questions and decisions to confirm

Every row below is a decision the specification’s author made on the requester’s behalf because `one_sentence_prompt.md` did not settle it. The normative rows assume the default; the reviews checked that the default is precise, not that it is what the requester wanted. Rows are the **D-*nn*** family (decisions to confirm). A row marked **confirm** is ratified by a human changing its status to **confirmed v1.*n***; a row the requester overturns becomes a **fix(speccheck)**: change with a version bump.

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-01	Implementation stack	Python 3.12 + uv, standard-library kernel, one HTTP client behind an extra	Go or Rust (single static binary; stack-neutral repos would not need a Python runtime); TypeScript (nearest to most agent tooling)	§10, C-02/C-03 type pins, K-10	requester / confirm
D-02	Test-result input contract	JUnit XML, consumed; the tool never runs tests	run the suite itself (simpler for users, but executes untrusted code and ties the tool to one runner); pytest’s own JSON report (richer, Python-only)	R-05, C-04, non-goal “no test execution”	requester / confirm
D-03	What counts as a citation	a literal ID token anywhere in a file, with <code>speccheck:ignore</code> opt-outs	a structured annotation (<code>@spec [R-07] (#sec-contracts)</code>) — precise but requires adoption; docstring-only citations — misses comments	C-01, R-03, R-04, R-27, the §0 limitation	requester / confirm
D-04	Family-T semantics	option (a): T ids are in scope, cited by their own test, and count in conformance	option (b): T ids declared but out of scope, reported separately — leaves conformance about R/C/I/K/E only	R-25, C-05 step 2b, C-07 metrics, T-53	requester / confirm (F-001 chose (a) on the reviewer’s recommendation)
D-05	Mixed judge verdicts under <code>--strict</code>	an ID with both ASSERTS and EXECUTES_ONLY edges is PASSING and passes strict	a MIXED status that fails strict — stricter, noisier	R-15, C-05 step 5, E-26	requester / confirm (F-003)
D-06	Judge question and vocabulary	one question per edge — does this test <i>assert</i> the behavior? — with ASSERTS / EXECUTES_ONLY / UNRELATED / UNKNOWN	a graded strength score (rejected: a number the model made up); judging code rather than tests (rejected: semantic analysis is a non-goal)	C-06, C-10, I-004, I-005	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-07	Judge request parameters	<code>temperature 0</code> , <code>max_tokens 4000</code> (v1.2; was 400), 30 s timeout, concurrency 4, budget unlimited, <code>--max-unknown 0.2</code>	any of these numbers; <code>max_tokens 400</code> was rejected after the first T-49 runs because thinking models (<code>qwen3:8b</code> , <code>gemma4:latest</code>) spend the budget on reasoning and are truncated before the verdict JSON, driving <code>unknown_rate</code> above <code>--max-unknown</code> ; at 4000 both pass T-49 three runs out of three. The other numbers remain defaults with no data behind them	C-06 , K-05 , K-06 , K-11 , K-12	requester / <code>max_tokens</code> confirmed; the rest remain confirm
D-08	Judge instruction text	the C-10 text as written	any rewording; the text is a first draft whose only evidence is T-49	C-10 , R-26 , T-49	requester / confirm after the first T-49 run
D-09	Input limits	files over 2 MiB skipped; NUL byte in the first 8 KiB means binary; IDs 1–3 digits	larger cap, or none; MIME sniffing; 4-digit IDs	K-02 , K-04 , E-10 , E-29	requester / confirm
D-10	LLM endpoint shape	an OpenAI-compatible <code>chat/completions</code> body, provider-agnostic	a specific vendor SDK (simpler, vendor-locked); a local-model-only path	C-06 , C-09	requester / confirm
D-11	Fixture and self-check layout	<code>fixtures/target/</code> with <code>golden/</code> , byte-copied into the wheel as <code>speccheck/_selfcheck/</code> entirely	package only the goldens and regenerate the fixture; skip the self-check	§10 , T-43 , T-46 , T-60	requester / confirm
D-12	Report file names and location	<code>SPEC_CONFORMANCE_REPORTS/</code> and <code>speccheck.json</code> under <code>--out</code> , default <code>.</code>	<code>REPORTS/</code> directory; a single JSON with Markdown derived by a separate renderer	§3.3 , C-07 , C-08 , E-18	requester / confirm
D-13	Python test-case delimitation	<code>def test_*</code> at module level; <code>test*</code> methods in <code>Test*</code> classes and <code>*TestCase</code> subclasses; nested classes excluded	honor <code>pytest.ini</code> <code>python_functions / python_classes</code> ; collect via pytest itself (rejected: runs code)	C-03 , E-28 , T-56	requester / confirm (F-007, F-102)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-14	Reference machine for K-08	named in <code>SPEC_BUILD_REPORT.md</code> at build time	a CI runner with a generous bound; no performance constraint at all	K-08 , T-51	build owner / open

None of the fourteen was raised as a question before v1.1; each was decided and reviewed for precision only. That is the defect this section corrects: a specification can be implementation-grade and still not be what was asked for.

Revision history

Version	Change
v0.1	Initial draft from <code>one_sentence_prompt.md</code> , for <code>spec-review</code> .
v0.2	P0 of <code>SPEC_REVIEW_REPORT.md</code> applied: F-001 T-family semantics (C-01 , C-05 step 2b, R-25 , E-25 , T-53 , T-48); F-002 scan exclusions and <code>--out</code> inside <code>--root</code> (C-03 , §5.1, E-09 , E-23 , T-36); F-003 status is the sole input to <code>--strict</code> (R-15 , §5.4, C-05 , E-26 , T-27 , T-39); F-004 LLM body, response path, and normative instruction text (C-06 , C-10 , R-26 , C-07 <code>judge_prompt_sha256</code> , T-33 , T-54); F-005 parametrized JUnit names (C-04 , C-07 <code>results</code> , E-24 , T-52). P1 (F-006..F-013) and P2 (F-014..F-017) remain open.
v0.3	P1 of <code>SPEC_REVIEW_REPORT.md</code> applied: F-006 longest-suffix classname join (C-04 , E-27 , T-58); F-007 <code>*TestCase</code> classes and <code>async def</code> (C-03 , E-28 , T-56); F-008 binary rule and no symlinks (C-03 , K-02 , K-03 , E-29 , E-30 , T-13); F-009 row/cell/heading declaration grammar (C-01 , E-31 , T-55); F-010 one-line ASCII summary with regex, <code>UTF-8</code> stdout (§5.1, R-29 , T-44); F-011 packaged self-check fixture in a temp dir, I-001 exemption, §9.8 reference (§5.1, I-001 , §10, T-43); F-012 <code>--strict --judge llm</code> requires an available judge and <code>unknown_rate ≤ --max-unknown</code> (R-28 , K-11 , C-07 , §5.4, E-32 , T-59); F-013 <code>speccheck:ignore</code> markers and the stated literal-citation limitation (§0, C-01 , R-27 , E-33 , T-57). P2 (F-014..F-017) remains open.
v0.4	P2 of <code>SPEC_REVIEW_REPORT.md</code> applied: F-014 logging default <code>ERROR</code> , Notes at <code>INFO</code> , nothing at <code>WARNING</code> (§5.3, T-42); F-015 in-memory render, temp-and-rename dual write, cleanup on failure, <code>--out</code> creation permitted (§3.1, §3.3, I-001 , E-18 , T-45); F-016 <code>judge_available</code> null/true/false rule, <code>by_family</code> always six keys, <code>by_status</code> seven in-scope statuses, ID cell struck (C-07 , C-08 , T-34 , T-35); F-017 R-24 scoped to deterministic statuses plus recorded verdicts, T-49 per-run not pooled, K-08 / T-51 recorded on a named reference machine via <code>tools/bench.py</code> , <code>SPEC.md</code> decoded <code>UTF-8-with-replacement</code> (R-24 , T-49 , K-08 , T-51 , §5.1, E-11 , T-01). No findings open.
v0.5	<code>FINAL_SPEC_REVIEW_REPORT.md</code> P1 and P2 applied: F-101 <code>join_name</code> strips from the first [, T-52 corrected; F-102 <code>test*</code> methods (no underscore) in recognized classes (C-03 , E-28 , T-56); F-103 temporaries get a per-run nonce, are excluded from scans, and leftovers are deleted first (§3.1, C-03 , E-34 , T-36 , T-45); F-104 R-10 passed-outcome clause, R-19 self-check exemption; F-105 empty <code>classname</code> join and <code>null unknown_rate</code> (C-04 , K-11 , E-36 , T-52 , T-59); F-106 JSON-then-Markdown rename order and the one permitted stale combination (§3.1, E-18 , T-45); F-107 inner self-check exit expectation, <code>_selfcheck/</code> byte-identical to <code>fixtures/target/</code> , drift guard T-60 (§5.1, §10); F-108 fence definition and fallback <code>classname</code> (C-01 , C-03 , T-05); F-109 line-numbered <code>source</code> in the judge request, marker fixtures in data files (C-06 , C-10 , §9); F-110 <code>--judge-concurrency</code> and <code>--judge-budget</code> (K-06 , K-12 , §5.1, E-35 , T-33 , T-61). No findings open.

Version	Change
v1.0	All eleven findings of the independent SPEC_v0.5_REVIEW_REPORT_by_QWEN.md applied (cited as Q- <i>nnn</i>): Q-001 <code>verdict</code> key always present, <code>null</code> when unjudged, em dash in Markdown (C-07, C-08, E-37, T-34, T-35); Q-002 <code>notes</code> sorted by code point (C-07, E-38, T-34); Q-003 exact in-process <code>--self-check</code> invocation with <code>--results</code> , goldens under <code>fixtures/target/golden/</code> (§5.1, §10, T-43, T-46, T-60); Q-004 <code>unavailable</code> takes precedence over <code>unknown_rate</code> (C-07, §5.1, E-32, T-59); Q-005 <code>skipped</code> count added to the summary line and regex (§5.1, T-44); Q-006 “first declaration wins” removed (C-01); Q-007 symlinked command-line roots resolved and descended, descent-time symlinks skipped (C-03, E-09, E-30); Q-008 fence closing pinned — same marker character, no info string, no cross-type closure (C-01, T-05); Q-009 ratios as quantized Decimals emitted with exactly four decimals, K-11 compares the quantized value (C-07, K-11, T-34); Q-010 K-05 timeout spans the full request lifecycle, K-12 deadline semantics with in-flight completion (K-05, K-12, E-35, T-61); Q-011 <code>name</code> is "" in JSON, (<code>file</code>) is Markdown-only, R-13 concerns data (C-07, C-08, E-13, T-35). No findings open.
v1.1	Added §12, <i>Open questions and decisions to confirm</i> : fourteen decisions the author took by default on the requester’s behalf (stack, results contract, citation rule, T-family option, mixed-verdict strict rule, judge question and parameters, instruction text, input limits, endpoint shape, fixture layout, report names, test delimitation, reference machine), each with its default, the alternatives rejected, the IDs it affects, and a <code>confirm</code> / <code>open</code> status. No normative row changed. The <code>spec-writing</code> skill now requires this section.
v1.2	D-07 partly resolved by the first T-49 runs: <code>max_tokens</code> raised from 400 to 4000 in the judge request body (C-06), the wire-format test updated to match (T-33), and the D-07 row records the evidence. Thinking models truncated at 400 before emitting the verdict JSON; at 4000 <code>qwen3:8b</code> and <code>gemma4:latest</code> pass T-49 three runs out of three. No other normative row changed.